Master's Thesis Proposal: Towards an Autonomous Lifecycle Agent for PDDL Domain Modelling

Roshan Gupta
Academic Supervisor: **Prof. Dr. Ilche Georgievski**
Institute of Architecture of Application Systems (IAAS), University of Stuttgart

1 Research Questions and Hypotheses

Main Research Question (MRQ): *How can an autonomous agentic framework integrate large language models (LLMs) with deterministic compilation and symbolic planning oracles to enable reliable PDDL domain model formulation, closed-loop repair, and continuous domain evolution without breaking existing domain solvability?*

1.1 Sub-Research Questions (RQs)

- **RQ1 (AST-Centric Domain Formulation):** What is the effect of decoupling relational semantic reasoning (using an Abstract Syntax Tree blueprint) from concrete PDDL syntax generation via deterministic compilation on syntactic correctness and plan validity compared to direct text generation?
- **RQ2 (Diagnostic Triage & Repair):** To what extent does layered diagnostic triage (categorizing errors into syntax, typing, precondition, effect, and reachability) reduce the repair iterations required to produce a valid plan compared to unstructured error feedback?
- **RQ3 (Domain Evolution & Solvability):** Can an autonomous agent evolve action capabilities in a classical PDDL domain in an incremental fashion while preserving reachability and solvability across a reference benchmark problem suite?
- **RQ4 (Model Sensitivity & Determinism):** How do different reasoning models (OpenAI GPT-4o and DeepSeek-Chat) perform under deterministic decoding (temperature = 0.0) with respect to structural reliability, plan efficiency, and computational cost?

1.2 Hypotheses

(H1) Generating models through a structured relational AST blueprint combined with a deterministic compiler will eliminate token-level syntax errors and achieve higher target plan validity than direct text generation.

(H2) Providing structured, layered diagnostic feedback will significantly reduce repair iterations (mean $k \leq 2$) compared to re-prompting with raw, unstructured planner error traces.

(H3) Enforcing reachability preservation across baseline problem suites during domain capability extension will allow the agent to synthesize new planning actions while maintaining complete solvability on reference problem instances in most domains.

(H4) Restricting LLM decoding to temperature 0.0 will maximize structural consistency, prevent stochastic token drift in formal logical formulations, and ensure reproducible experimental evaluation without compromising domain expressiveness.

2 Large Language Model Selection and Decoding Configuration

Model Selection: We evaluate two frontier language models: (1) **OpenAI GPT-4o** (`openai/gpt-4o`), a general-purpose model optimized for complex structured JSON-schema instruction following, and (2) **DeepSeek-Chat** (`deepseek/deepseek-chat`), an open-weight Mixture-of-Experts reasoning model tuned for code generation and formal symbolic tasks.

Temperature Configuration (temperature = 0.0): Classical PDDL is an exact mathematical language based on First-Order Logic that does not tolerate stochastic sampling noise. Setting temperature to 0.0 forces greedy argmax token selection, strictly enforces dictionary key adherence in Pydantic AST schemas, ensures scientific reproducibility across experimental runs, and guarantees that any repair is causally attributable to diagnostic feedback rather than random generation.

3 Benchmark Selection

To evaluate the framework across diverse structural topologies, all selected domains strictly adhere to classical PDDL 2.1 (`:strips` and `:typing`) from IPC (1998–2014) tracks and AutoPlanBench:

- **Closed-Loop Domain Formulation Benchmark (20 Domains):** *Logistics, Driverlog, Elevators, Zenotravel, Gripper, Floortile, Grid, Goldminer, Rovers, Blocksworld, Parking, Pegsol, Mystery, Barman, Pipesworld, Movie, Depot, Ferry, Satellite and Scanalyzer*
- **Continuous Domain Evolution Benchmark (10 Domains):** *Satellite, Depot, Gripper, Rovers, Barman, Floortile, Logistics, Pipesworld-Tankage, Driverlog, Ferry.*

All IPC benchmark domains are normalized to pure unit-cost STRIPS: `:action-costs` requirements and cost functions are stripped so that plan validity and solvability depend strictly on relational state-transition dynamics.

4 Experimental Baselines and Ablation Studies

4.1 Baselines and Ablations

Baselines: (1) **Baseline 1 (Zero-Shot Direct Prompting):** LLM generates complete PDDL domain and problem files in a single pass without intermediate representations or feedback loops. (2) **Baseline 2 (Unstructured Iterative Retry):** LLM generates raw PDDL text and is iteratively re-prompted with raw, unparsed planner error logs without structured triage.

Ablation Studies: To isolate individual component contributions, we compare the complete system against: (1) **Ablation 1 (No-AST / Direct Text Repair):** Receives diagnostic triage feedback but edits raw PDDL text directly, isolating the AST blueprint. (2) **Ablation 2 (No-Triage / AST Raw Feedback):** Uses the AST blueprint and compiler but receives raw unparsed error logs, isolating causal fault localization. (3) **Ablation 3 (No-Maintenance / Unconstrained Evolution):** Disables the Phase 4 Maintenance Verification during domain evolution to evaluate whether capability extensions compromise solvability across the reference benchmark problem suite. (4) **Full Proposed System:** Integrates AST blueprints, deterministic compilation, layered triage, and solvability preservation verification.

4.2 Layered Diagnostic Triage and Fault Taxonomy

The framework introduces a Five-Layer Causal Fault Taxonomy that replaces unparsed planner error dumps with structured diagnostic feedback:

(1) **Syntax & Grammar Validation (Layer 1):** Isolates token-level lexical errors, unmatched parentheses, and malformed PDDL keywords via the VAL parser.
(2) **Typing & Signature Alignment (Layer 2):** Detects predicate arity violations, parameter type mismatches, and cyclic type hierarchies during translation.
(3) **Precondition Bottlenecks (Layer 3):** Pinpoints over-constrained or contradictory preconditions that block action execution along state trajectories.
(4) **Action Effect Deficiencies (Layer 4):** Identifies missing add/delete effects, unretracted facts, and simultaneous assertions that invalidate state transitions.
(5) **Goal Reachability & Solvability Regressions (Layer 5):** Analyzes search-space deadlocks where goal conditions are causally disconnected, or detects reachability regressions across reference benchmark problem suites during domain evolution.

In the fault repair benchmark, each task instance contains a single isolated defect injected at a specific action schema. Ground-truth defect locations are labeled at the individual action-schema level (e.g., `domain.actions.pick-up`) to compute Top-1 Fault Localization Accuracy (FLA) and Mean Reciprocal Rank (MRR). Converting raw oracle errors into localized AST repair directives enables the diagnostic triage node to precisely modify faulty action schemas while preserving verified operators.

5 Evaluation Metrics and Symbolic Oracles

All metrics are evaluated fully automatically using symbolic oracles: the official **VAL** plan validator and the **Fast Downward** planning system. Fast Downward executes lazy greedy best-first search with the FF heuristic

(`lazy_greedy([ff()])`) under a search cutoff of **30 seconds** and 4 GB RAM, while VAL plan validation is enforced under a **15-second** timeout. Syntax Validity Rate (SVR) is determined deterministically by the official VAL `Parser` binary. Statistical significance for H1 and H2 is verified using the paired Wilcoxon signed-rank test and McNemar's test.

Metric Name	Formal Definition / Formula	Operational Purpose	Mapped RQ & H
Syntax Validity Rate (SVR)	SVR = $\frac{N_{\text{valid}}}{N_{\text{total}}} \times 100\%$ verified via VAL `Parser`.	Verifies that the AST compiler eliminates token-level syntax crashes (e.g., missing parentheses, colons).	RQ1 (H1)
Target Plan Validity (TPV)	TPV = True $\iff$ FD(P) = 0 $\wedge$ VAL(π_{gen}) = Valid; else False.	Measures whether the candidate model is operationally solvable and self-consistent under Fast Downward and VAL.	RQ1, RQ3, RQ4 (H1, H3, H4)
Plan Transfer Solvability (PTS)	PTS = True $\iff$ VAL($D_{\text{gt}}, P, \pi_{\text{gen}}$) = Valid; else False.	Non-circular semantic soundness certificate; validates whether plan π_{gen} executes validly on ground-truth domain D_{gt}.	RQ1, RQ4 (H1, H4)
Reference Suite Coverage (Cov_{ref})	$\text{Cov}_{\text{ref}} = \frac{N_{\text{solved}}}{N_{\text{total}}} \times 100\%$ on reference problem suite Π_{ref}.	Measures reachability preservation; detects solvability regressions where action modifications satisfy the capability extension but break solvability on reference problem instances Π_{ref}.	RQ3 (H3)
Fault Localization Accuracy (FLA)	FLA = $\mathbb{I}(\text{Top-1}(\hat{a}) = a_{\text{gold}})$; MRR = $\frac{1}{\text{rank}(a_{\text{gold}})}$.	Measures the accuracy of the diagnostic triage node in localizing the exact faulty action schema prior to repair.	RQ2 (H2)
Macro Structural F1	Macro_F1 = $\frac{\sum \text{Structural_F1}(op_i, op_j)}{N_{\text{ground_truth}}}$ via Kuhn-Munkres matching.	Measures structural operator fidelity against gold-standard models without penalizing synonymous action naming.	RQ1, RQ4 (H1, H4)
STRIPS Component F1 (Pre, Add, Del)	F1 = $\frac{2 \cdot P \cdot R}{P+R}$ evaluated independently on action preconditions, add effects, and delete effects.	Isolates exact causal failure locations (missing preconditions vs. contradictory effects) to guide repair.	RQ1, RQ2 (H1, H2)
Plan Length / Cost (|Plan|)	Number of sequential action steps in the validated plan: |Plan| = n.	Measures plan efficiency and confirms that solutions avoid redundant action loops across models.	RQ4 (H4)
Feedback Iterations (k)	Integer number of repair cycles required to reach validity: $k \in \{1, 2, \ldots, N_{\max}\}$ ($k = 1$ is Zero-Shot).	Measures closed-loop repair efficiency and the effectiveness of the diagnostic triage node.	RQ2 (H2)
Execution Latency & Cost	Time = $t_{\text{end}} - t_{\text{start}}$ (seconds); Cost = $\sum$(Tokens $\times$ Price) (USD).	Measures computational overhead, API latency, and practical deployment feasibility across models.	RQ4 (H4)

Table 1: Comprehensive experimental evaluation metrics, formal definitions, operational purposes, and mapping to research questions and hypotheses.

6 Relation to Thesis Objectives

This proposal is structured to map directly into the planned chapters of the Master's thesis:

(1) **Chapter 1 (Introduction):** Outlines real-world motivations, LLM-based planning limitations, MRQ, sub-questions (RQ1–RQ4), and hypotheses (H1–H4).
(2) **Chapter 2 (Theoretical Background):** Formally defines state-transition systems, PDDL 2.1 syntax, Fast Downward search, VAL validation, and greedy decoding.
(3) **Chapter 3 (Related Work):** Surveys action model learning, LLM-modulo systems, and domain maintenance frameworks.
(4) **Chapter 4 (System Architecture):** Details the agent core, Pydantic AST blueprints, deterministic compiler, triage hierarchy, and solvability preservation verification.
(5) **Chapter 5 (Technical Implementation):** Documents LangGraph state machines, Pydantic schemas, oracle subprocess wrappers, and OpenRouter integration.
(6) **Chapter 6 (Experimental Evaluation):** Evaluates 20 domain formulation benchmarks and 10 evolution domains against baselines and ablations using Table 1.
(7) **Chapter 7 (Conclusion and Future Work):** Synthesizes findings, confirms support for hypotheses H1–H4,

and discusses future research.